

Suppose a modern operating system supports multiple users and processes with shared resources like CPU, memory, and I/O devices.

(a) Classify different types of operating systems suitable for this scenario and justify your choice.

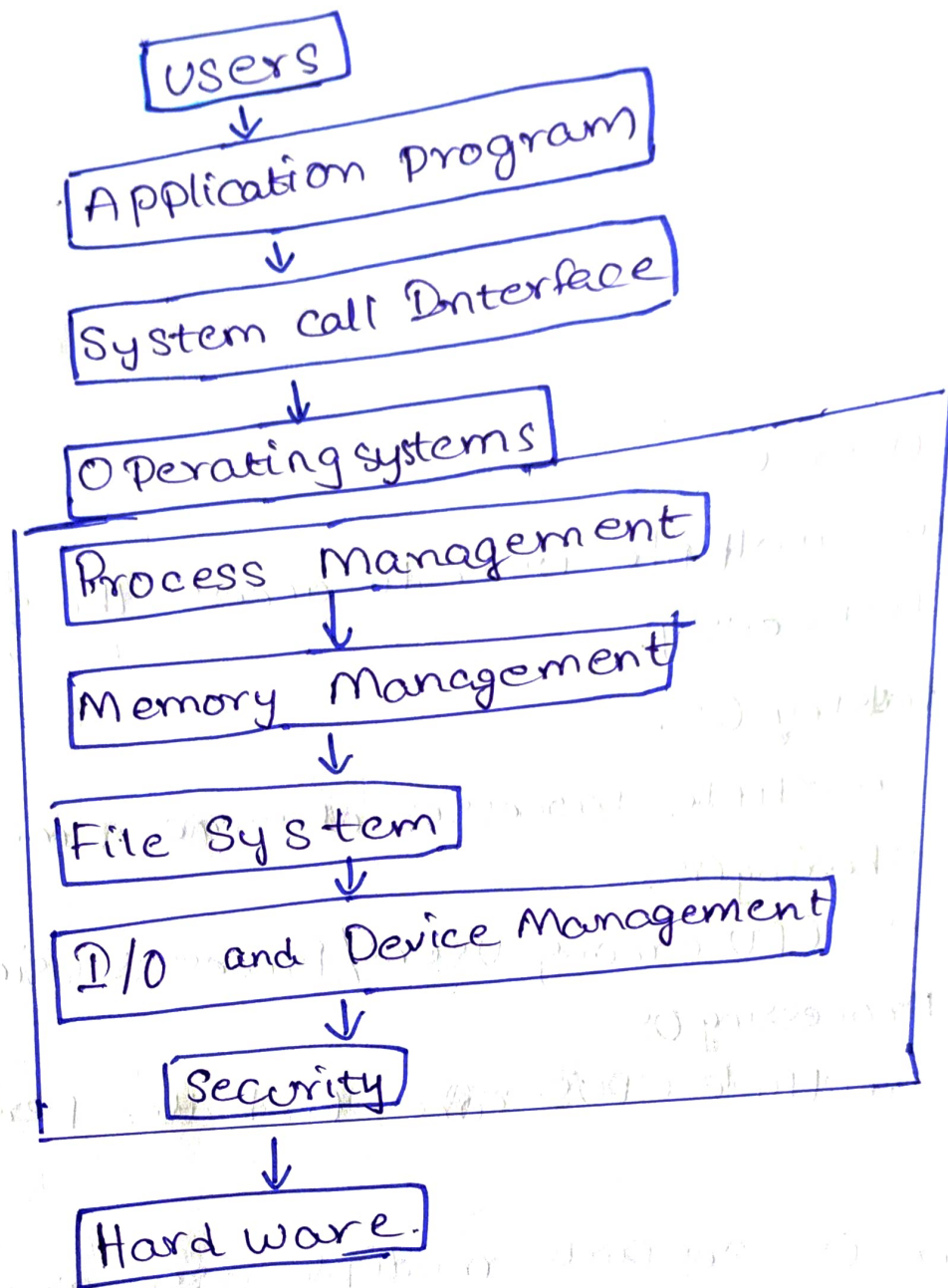
Suitable OS Types:

- * **Multi user OS:**
Allows multiple user to access the system simultaneously.
- * **Multitasking OS:**
Runs multiple processes at same time.
- * **Time sharing OS:**
Shares CPU among users/Processes using time slices.
- * **Multi processing OS:**
Uses multiple CPU's cores for better performance.

Justification:

Modern OS supports multiple users and processes while efficiently sharing CPU, memory, and I/O resources.

(b) Draw the structure of an operating system showing how resource are managed.



(C) Write a sequence of system calls required by to create and execute a new process in such environment.

1. `fork()` - Creates a child Process
2. `exec()` - loads and executes a new program
3. `wait()` - Parent waits for child to finish
4. `exit()` - Child terminate execution.

Sequence:

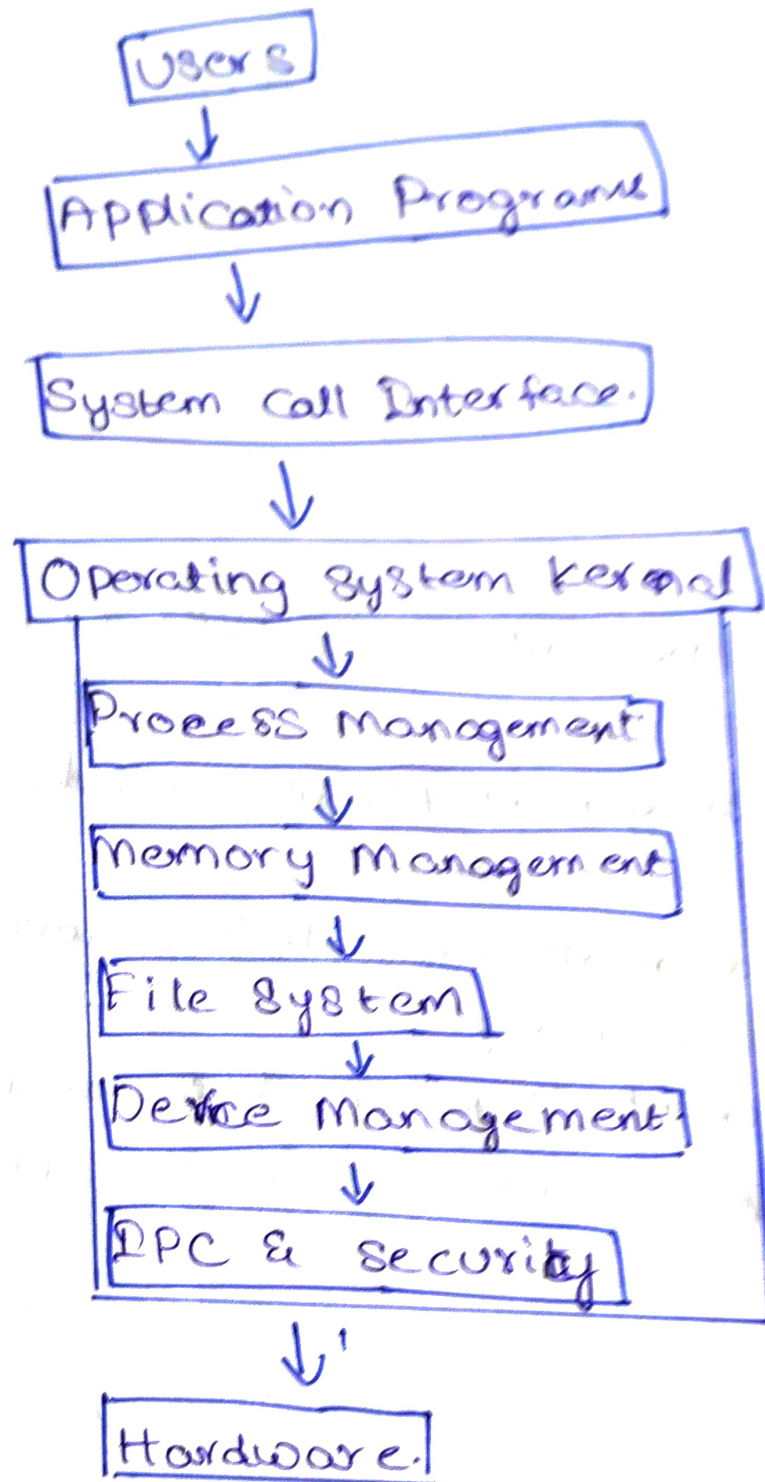
`fork()` $\rightarrow$ `exec()` $\rightarrow$ `wait()` $\rightarrow$ `exit()`

Consider a scenario where an Operating system must manage process scheduling, memory allocation, and device communication efficiently.

(b) Explain the major functions of an OS in handling these tasks.

- * **Process Management:** Creates, Schedules, and terminates processes.
- * **Memory Management:** Allocates and deallocate memory, manages virtual memory.
- * **Device Management:** Controls I/O devices through drivers and buffering.
- * **File Management:** Organizes files and directories.
- * **Security :** Protects data and controls user access.

(b) Draw a block diagram representing OS components and their interactions.



(c) write a system call examples for memory allocation and device & management.

memory Allocation:

brk()

sbrk()

mmap()

munmap()

Device management:

Open()

read()

write()

close()

lseek()